

Bootcamp Databricks

Engenharia de Dados dentro da Plataforma Data & AI

I&D Capgemini Brasil, Setembro/2026

Agenda

Aulas	Assunto
Dia 1 – 14/09/2026	Fundamentos de Engenharia de dados e Databricks
Dia 2 – 15/09/2026	SQL, bancos, tabelas e Delta Lake
Dia 3 – 16/09/2026	Unity Catalog, schemas, volumes e governança
Dia 4 – 17/09/2026	Ingestão de dados: Batch, Incremental, Auto Loader
Dia 5 – 18/09/2026	ETL vs ELT & transformação arquitetura medalhão
Dia 6 – 21/09/2026	Modelagem de dados para Analytics
Dia 7 – 22/09/2026	Processamento distribuído com Spark no Databricks
Dia 8 – 23/09/2026	Streaming e processamento contínuo
Dia 9 – 24/09/2026	Orquestração, Jobs, DataOps e boas práticas
Dia 10 – 25/09/2026	Qualidade, observabilidade, arquitetura e desafio final

Dia 5 - ETL vs ELT & transformação arquitetura medalhão

Da Bronze bruta à Gold pronta para negócio

I&D Capgemini Brasil, Setembro/2026

ETL



Transform before load

VS

ELT



Load first, transform after



Unity Catalog



Governança



Segurança



Compartilhamento

FONTES DE DADOS

- Cloud Storage (S3 / ADLS / GCS)
- Bancos de Dados (OLTP / RDBMS)
- Streaming / Kafka
- APIs
- Arquivos / Logs (CSV, JSON, Parquet)
- SaaS / Salesforce



BRONZE DADOS BRUTOS

Ingestão rápida e íntegra como chegou da fonte



AUTO LOADER

- Listagem incremental
- ✓ CDC e novas detecções
- ✓ Schema evolution
- ✓ Processamento contínuo

DELTA TABLE



SILVER DADOS TRATADOS

Limpeza, padronização e enriquecimento inicial



TRANSFORMAÇÕES

- ✓ Regras de negócio
- ✓ Qualidade de dados
- ✓ Conformidade de schema
- ✓ Joins e enriquecimentos

DELTA TABLE



GOLD DADOS DE NEGÓCIO

Modelagem, agregações e dados prontos para consumo



DATA PRODUCTS

- ✓ Modelos dimensionais
- ✓ KPIs e dashboards
- ✓ APIs e Data Sharing
- ✓ Features para ML/IA

DELTA TABLE

CONSUMO

- BI / Dashboards (Power BI, Tableau)
- Analytics (Ad-hoc / Self Service)
- Machine Learning (Modelos / Features)
- APIs / Aplicações
- Data Sharing (Interno / externo)



QUALIDADE DE DADOS

- ✓ Regras e validações
- ✓ Observabilidade
- ✓ Alertas e métricas



LINHAGEM E METADADOS

- ✓ Rastreamento ponta a ponta
- ✓ Impacto e dependências
- ✓ Catalogo unificado



SEGURANÇA E ACESSO

- ✓ Unity Catalog
- ✓ Controle de privilégios
- ✓ Row / Column level



AUDITORIA E CONFORMIDADE

- ✓ Logs e auditoria
- ✓ Retenção e compliance
- ✓ Políticas e tags



DATABRICKS
LAKEHOUSE PLATFORM



Notebooks



Jobs / Workflows



Delta Lake



Unity Catalog



Auto Loader



Photon Engine



MLflow



Feature Store



Data Sharing

Objetivos do Dia 5

O aluno aprende o desenho clássico de pipeline lakehouse.

- Diferenciar ETL e ELT no contexto de Data Warehouse, Data Lake e Lakehouse.
- Entender a arquitetura medalhão: Bronze, Silver e Gold.
- Aplicar padrões de transformação: limpeza, padronização, deduplicação e derivação.
- Usar Delta Lake para MERGE, idempotência e reprocessamento controlado.
- Construir pipeline funcional Bronze → Silver → Gold e publicar tabela analítica.

Mensagem central

Transformação não é “arrumar dado”. É criar contratos progressivos de qualidade.

Entregável

Pipeline Bronze/Silver/Gold funcional com tabela Gold de consumo analítico.

Recap do Dia 4: Bronze populada

Agora a matéria-prima existe. Falta transformar com critério.

Dia 4

Arquivos ingeridos para Bronze com Auto Loader.

Dia 5

Limpeza, tipagem, regras e agregações.

Dia 6

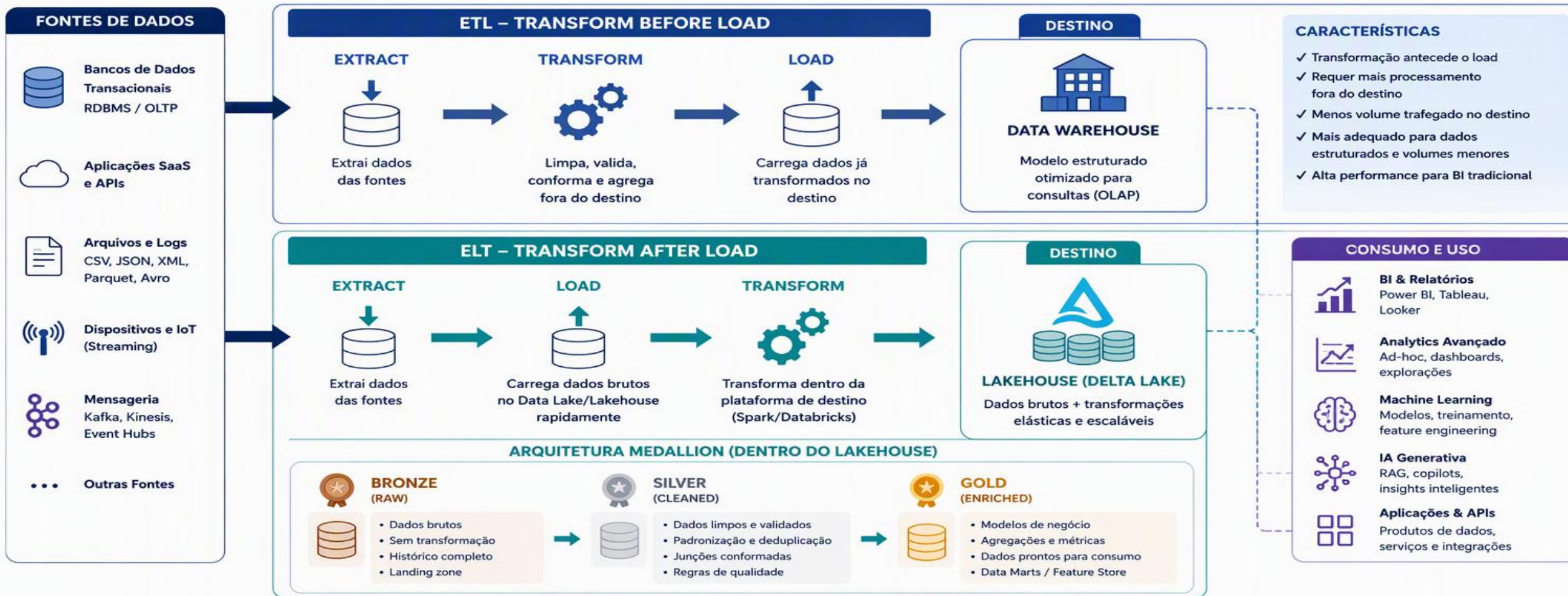
Modelagem para Analytics.

```
capgemini_academy.bronze.pedidos_autoloader  
→ capgemini_academy.silver.pedidos_autoloader  
→ capgemini_academy.gold.receita_diaria
```

Parte 1 - ETL vs ELT

ETL vs ELT – Padrões de Transformação

Duas formas de levar dados das fontes até o consumo com qualidade, governança e escala



COMPARATIVO RÁPIDO

Aspecto	ETL	ELT
Onde transforma	Fora do destino	Dentro do destino
Velocidade de load	Mais lenta	Mais rápida
Volume suportado	Menor	Muito alto
Flexibilidade	Menor	Maior
Cenário ideal	Dados estruturados, volumes menores	Grandes volumes, semi/ não estruturados

QUANDO USAR?

Use ETL quando:

- Dados altamente estruturados
- Regras de negócio complexas fora do destino
- Conformidade exige staging antes de armazenar
- Volumes moderados

Use ELT quando:

- Grandes volumes de dados
- Dados semi/ não estruturados
- Necessidade de reprocessamento e exploração
- Lakehouse como plataforma central

VS

BOAS PRÁTICAS DE INGESTÃO



Governança com Unity Catalog

Catálogos, schemas, Caálogos, schenae, permissões e lineage



Qualidade de Dados

Validações, regras, expectations e observabilidade



Processos Idempotentes

Permitem reprocessar sem duplicidade e com segurança

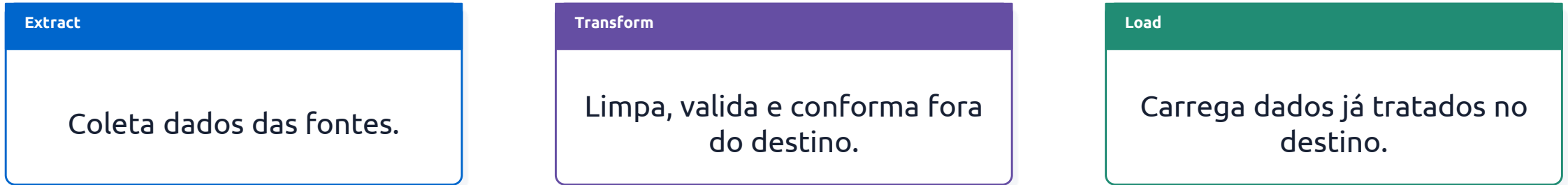


Monitoramento e Alertas

Volumes, latência, erros e inferência de schema

ETL tradicional

Transforma antes de carregar no destino analítico.



- Adequado quando o destino não deve receber dados brutos.
- Comum em arquiteturas tradicionais de Data Warehouse.
- Pode limitar reproprocessamento se o dado bruto não for persistido.

ELT no Lakehouse

Carrega primeiro e transforma dentro da plataforma de dados.



- Permite preservar histórico bruto para auditoria e reproprocessamento.
- Combina armazenamento escalável com processamento Spark/SQL.
- Conecta naturalmente com arquitetura medalhão.

Critério prático: quando escolher cada padrão

Não existe "melhor" em abstrato; existe melhor para o contexto.

Critério	ETL	ELT
Transforma onde?	Antes do destino	Dentro da plataforma de destino
Dado bruto no destino?	Geralmente não	Sim, na Bronze
Reprocessamento	Depende da retenção fora do destino	Pode reprocessar a partir da Bronze
Flexibilidade	Menor para exploração posterior	Maior para evolução de regras
Aderência ao Lakehouse	Menor	Maior

Parte 2 - Arquitetura medalhão



1. FONTES



2. INGESTÃO



3. TRANSFORMAÇÃO



4. MODELAGEM



5. CONSUMO

FONTES DE DADOS



Sistemas Transacionais (ERP, CRM, POS)



Aplicações SaaS (Salesforce, etc.)



APIs e Serviços



Arquivos (CSV, JSON, Parquet)



Streaming / Kafka



Dispositivos / IoT

...



BRONZE

DADOS BRUTOS (RAW)

Ingestão rápida e completa como recebido das fontes

Exemplo de tabela Bronze

bronze.sales.raw_orders				
order_id	customer_id	product_id	amount	ingest_ts
1001	C001	P10	120.50	2026-07-27 10:15:21
1002	C002	P15	75.00	2026-07-27 10:15:22
1003	C003	P10	220.00	2026-07-27 10:15:23
...	...	...	...	...

- ✓ Dados brutos e imutáveis
- ✓ Mantém histórico completo (append-only)
- ✓ Rastreabilidade e auditabilidade
- ✓ Schema on-read



DELTA LAKE



QUALITY GATE

- ✓ Schema Validation
- ✓ Expectations (Null, Range, Formato)
- ✓ Deduplicação básica
- ✓ Quarentena de falhas



Registros inválidos



REJECTS / QUARENTENA



SILVER

DADOS TRATADOS (CLEAN)

Dados limpos, deduplicados e padronizados

Exemplo de tabela Silver

silver.sales.orders_clean				
order_id	customer_id	product_id	amount	order_ts
1001	C001	P10	120.50	2026-07-27 10:15:21
1002	C002	P15	75.00	2026-07-27 10:15:22
1004	C004	P20	199.90	2026-07-27 10:15:24
...	...	...	...	...

- ✓ Deduplicação e validações avançadas
- ✓ Padronização de tipos e formatos
- ✓ Enriquecimento e conformação
- ✓ Dados prontos para uso analítico detalhado



DELTA LAKE

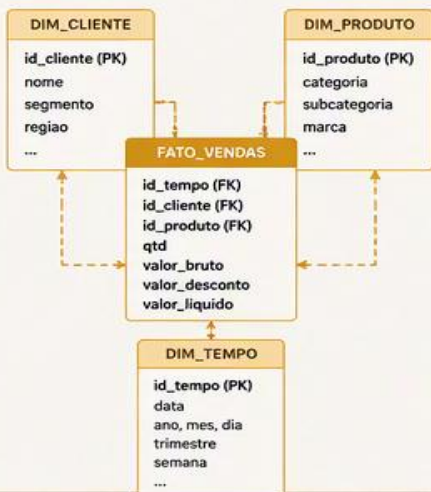


GOLD

DADOS DE NEGÓCIO (AGG/DIM)

Modelo dimensional e agregações para Analytics e BI

Exemplo de modelo dimensional



DELTA LAKE

CONSUMO / CASOS DE USO



Dashboards & BI (Power BI, Tableau)



Analytics Ad-hoc



Machine Learning & Data Science



Data Apps & APIs



Relatórios Operacionais

GOVERNANÇA E BASE OPERACIONAL (ATRAVESSA TODAS AS CAMADAS)



Unity Catalog (catálogo, schemas, permissões)



Linagem & Metadados Rastreabilidade ponta a ponta



Segurança RBAC, ABAC, Masking



Qualidade Contínua Monitoramento de regras e SLAs



Observabilidade Logs, métricas e alertas



DataOps CI/CD, versionamento, testes



Ambientes DEV • QA • PRD Isolamento e promoção

LEGENDA

- Fluxo de dados
- > Validação / Controle
-> Fluxo de rejeição

FLUXO RESUMIDO DO DADO



Fontes



Landing (Volume)



Auto Loader (Incremental)



Bronze (Raw)



Transformações (Silver)



Gold (Dim / Agg)



Consumo

BENEFÍCIOS DA MEDALLION ARCHITECTURE

- ✓ Qualidade crescente dos dados
- ✓ Reutilização e padronização
- ✓ Governança e segurança integradas
- ✓ Performance para Analytics e IA
- ✓ Escalável e resiliente

Bronze: raw, auditável e reprocessável

Bronze deve preservar o máximo possível do que veio da fonte.

O que entra

Dados brutos, metadados de ingestão e origem.

O que evitar

Regras de negócio complexas ou filtros irreversíveis.

Por que existe

Auditoria, lineage, histórico e reprocessamento.

- Bronze como camada de dados raw provenientes de sistemas externos.
- Use colunas técnicas como `_ingested_at`, `source_file` e `batch_id`.
- Bronze é base de reconstrução quando regras mudam.

Silver: dados limpos, validados e conformados

É onde a engenharia de transformação realmente acontece.

Limpeza

Tratamento de nulos, tipos, formatos e valores inválidos.

Conformidade

Deduplicação, padronização e chaves consistentes.

Uso

Base refinada para análises detalhadas e Gold.

- Silver como camada de limpeza e validação.
- Silver pode aplicar deduplicação, validações e joins de referência.
- Decisões de regra devem ficar explícitas e testáveis.

Gold: dados de negócio prontos para consumo

Gold deve ser orientada a métricas, agregações e modelos de consumo.

Agregações

Receita diária, volume por categoria, KPIs.

Modelagem

Fatos, dimensões, data marts e features.

Consumo

BI, analytics, ML, APIs e produtos de dados.

- Gold como camada para modelagem dimensional e agregação.
- Gold não deve ser apenas “mais limpa”; deve responder perguntas de negócio.
- O Dia 6 aprofunda modelagem para Analytics.

Patterns de transformação Bronze → Silver

Transformações típicas da camada Silver.

Tipagem

Cast para tipos corretos e compatíveis.

Nulos

Filtrar, imputar ou segregar inválidos.

Deduplicação

Eliminar duplicidades por chave e regra.

Padronização

Datas, formatos, nomes e domínios.

```
CREATE OR REPLACE TABLE capgemini_academy.silver.pedidos_autoloader AS
SELECT
  CAST(pedido_id AS BIGINT) AS pedido_id,
  TRIM(cliente_id) AS cliente_id,
  CAST(quantidade AS INT) AS quantidade,
  CAST(valor_total AS DECIMAL(10,2)) AS valor_total,
  TO_TIMESTAMP(data_pedido) AS data_pedido
FROM capgemini_academy.bronze.pedidos_autoloader
WHERE pedido_id IS NOT NULL
  AND quantidade > 0;
```

Patterns de transformação Silver → Gold

Gold materializa perguntas de negócio em tabelas de consumo.

Métricas

Somas, médias, contagens e taxas.

Dimensões

Datas, produtos, clientes e hierarquias.

Consumo

Tabelas otimizadas para BI e analytics.

```
CREATE OR REPLACE TABLE capgemini_academy.gold.receita_diaria AS
SELECT
  DATE(data_pedido) AS data_pedido,
  COUNT(DISTINCT pedido_id) AS pedidos,
  SUM(quantidade) AS itens_vendidos,
  SUM(valor_total) AS receita_total,
  ROUND(AVG(valor_total), 2) AS ticket_medio
FROM capgemini_academy.silver.pedidos_autoloader
GROUP BY DATE(data_pedido);
```

Parte 3 - Idempotência, MERGE e reproprocessamento

Idempotência: executar de novo sem duplicar estrago

Um pipeline deve permitir reexecução segura.

Problema

O mesmo lote pode ser executado mais de uma vez.

Risco

Duplicidade, métricas infladas e perda de confiança.

Resposta

Chaves, MERGE, deduplicação e controle de processamento.

- Idempotência é objetivo de engenharia: mesma entrada, mesmo resultado final.
- Append puro costuma ser insuficiente quando há atualizações ou reproprocessamento.
- MERGE ajuda em padrões de upsert sobre tabelas Delta.

MERGE INTO: upsert controlado

Delta Lake permite combinar fonte e destino com INSERT, UPDATE e DELETE.

```
MERGE INTO capgemini_academy.silver.pedidos_autoloader AS target
USING capgemini_academy.bronze.pedidos_autoloader AS source
ON target.pedido_id = source.pedido_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

- MERGE pode fazer upsert de source table, view ou DataFrame em uma target Delta Lake table.
- A referência MERGE INTO descreve que a instrução mescla updates, insertions e deletions baseados em uma fonte em uma tabela Delta alvo.
- Use MERGE quando houver chave de negócio e necessidade de atualizar registros existentes.

Schema enforcement: tabela é contrato

Transformação deve respeitar schema, tipos e regra de escrita.

Enforcement	INSERT	MERGE
Valida schema na escrita em tabelas Delta.	Colunas inseridas devem existir na tabela alvo.	UPDATE/INSERT devem respeitar colunas da tabela alvo.

- Databricks valida qualidade reforçando schema on write em tabelas Delta.
- Schema enforcement não se aplica a tabelas usando formatos não Delta, como CSV ou JSON em cloud storage.
- Silver deve ter contrato mais forte que Bronze.

Reprocessamento: quando a regra muda

Bronze permite reconstruir Silver/Gold sem voltar à fonte.

Cenário

Regra de negócio muda ou bug é descoberto.

Ação

Reprocessar Silver/Gold a partir da Bronze preservada.

Controle

Versionar código, registrar execução e validar saída.

- O plano interno prevê rastreabilidade e dados auditáveis no pipeline Bronze → Silver → Gold.
- Reprocessamento deve ser explícito, testável e repetível.
- Não use PRD como laboratório de tentativa e erro.

Parte 4 - Demo e laboratório

Demo guiada: Bronze → Silver → Gold

Day 4 Hands-on Lab – Bronze → Silver → Gold

Cluster

Python

Schedule

+ New

Workspace

Recent

Catalog

Workflows

Compute

SQL

Data Engineering

Machine Learning

Marketplace

BRONZE (Raw)
ops.bronze.pedidos

pedido_id	cliente_id	produto_id	quantidade	preco	data_pedido
1001	C001	P10	2	120.00	2026-07-27 10:15:23
1002	C002	P20	1	250.00	2026-07-27 11:02:10
1003	C001	P10	NULL	120.00	2026-07-27 11:20:00
1004	NULL	P30	5	50.00	2026-07-27 11:45:10
1005	C003	P40	-1	300.00	2026-07-27 12:01:33
...					

Cloud Storage / Landing (Volume)
/Volumes/landing/pedidos/

SILVER (Clean)
ops.silver.pedidos_clean

pedido_id	cliente_id	produto_id	quantidade	preco	data_pedido
1001	C001	P10	2	120.00	2026-07-27 10:15:23
1002	C002	P20	1	250.00	2026-07-27 11:02:10
1006	C004	P10	3	120.00	2026-07-27 12:30:15
1007	C003	P30	5	50.00	2026-07-27 13:10:22
1008	C005	P40	1	300.00	2026-07-27 13:22:11
...					

Deduplicação Tratamento nulos Regras de negócio Padronização

GOLD (Aggregated)
ops.gold.receita_diaria

data_pedido	receita_total	pedidos	itens_vendidos	ticket_medio
2026-07-27	1,840.00	5	12	368.00
2026-07-28	2,350.00	6	15	391.67
2026-07-29	1,950.00	4	10	487.50
...				

Agregação KPIs Pronto para BI / ML

1 LER BRONZE – AUTO LOADER (CSV)

Python

```
1 checkpoint_path = "/Volumes/checkpoints/pedidos_bronze"
2 schema_path = "/Volumes/schemas/pedidos_bronze"
3
4 df_raw = (spark.readStream
5   .format("cloudFiles")
6   .option("cloudFiles.format", "csv")
7   .option("cloudFiles.inferColumnTypes", "true")
8   .option("cloudFiles.schemaLocation", schema_path)
9   .load("/Volumes/landing/pedidos/"))
10
11 (df_raw.writeStream
12   .option("checkpointLocation", checkpoint_path)
13   .trigger(availableNow=True)
14   .toTable("ops.bronze.pedidos"))
```

Dados na tabela Bronze (amostra)

pedido_id	cliente_id	produto_id	quantidade	preco	data_pedido_ingest_ts
1001	C001	P10	2	120.00	2026-07-27 14:00:01
1002	C002	P20	1	250.00	2026-07-27 14:00:01
1003	C001	P10	NULL	120.00	2026-07-27 14:00:01
...					

2 BRONZE → SILVER (LIMPEZA E PADRONIZAÇÃO)

SQL

```
1 CREATE OR REPLACE TABLE ops.silver.pedidos_clean AS
2 SELECT
3   CAST(pedido_id AS BIGINT)      AS pedido_id,
4   TRIM(cliente_id)              AS cliente_id,
5   TRIM(produto_id)              AS produto_id,
6   CASE WHEN CAST(quantidade AS INT) > 0
7     THEN CAST(quantidade AS INT) ELSE NULL END AS quantidade,
8   CAST(preco AS DECIMAL(12,2)) AS preco,
9   TO_TIMESTAMP(data_pedido)     AS data_pedido,
10  CURRENT_TIMESTAMP()           AS silver_ingest_ts
11 FROM ops.bronze.pedidos
12 WHERE cliente_id IS NOT NULL
13 AND TRIM(cliente_id) <> ''
14 AND CAST(CAST(quantidade AS INT) > 0
15 AND CAST(preco AS DECIMAL(12,2)) >= 0;
```

Dados na tabela Silver (amostra)

pedido_id	cliente_id	produto_id	quantidade	preco	silver_ingest_ts
1001	C001	P10	2	120.00	2026-07-27 10:05:18
1002	C002	P20	1	250.00	2026-07-27 11:03:18
1006	C004	P10	3	120.00	2026-07-27 12:05:18
...					

3 SILVER → GOLD (AGREGAÇÃO DIÁRIA)

PySpark

```
1 from pyspark.sql import functions as F
2
3 df_daily = (spark.table("ops.silver.pedidos_clean")
4   .withColumn("valor_total", F.col("quantidade") * F.col("preco"))
5   .groupBy(F.to_date("data_pedido").alias("data_pedido"))
6   .agg(
7     F.sum("valor_total").alias("receita_total"),
7     F.countDistinct("pedido_id").alias("pedidos"),
8     F.sum("quantidade").alias("itens_vendidos"),
8     F.round(F.avg("valor_total"), 2).alias("ticket_medio")
9   )
10 )
11 df_daily.write.mode("overwrite").saveAsTable("ops.gold.receita_diaria")
```

Dados na tabela Gold (amostra)

data_pedido	receita_total	pedidos	itens_vendidos	ticket_medio
2026-07-27	1,840.00	5	12	368.00
2026-07-28	2,350.00	6	15	391.67
2026-07-29	1,950.00	4	10	487.50
...				

VALIDAÇÃO & QUALITY CHECKS

✓

Schema detectado e armazenado

✓

Arquivos novos processados incrementalmente

✓

Nulos críticos tratados

✓

Valores inválidos filtrados

✓

Deduplicação aplicada

✓

Dados consistentes entre camadas

✓

Agregações conferidas

✓

Volumes e checkpoints gerenciados pelo Unity Catalog

COMANDOS ÚTEIS

>

DESCRIBE TABLE ops.bronze.pedidos;

>

DESCRIBE HISTORY ops.silver.pedidos_clean;

>

SELECT * FROM ops.gold.receita_diaria ORDER BY data_pedido DESC;

LINHAGEM DE DADOS (DATA LINEAGE)

Fonte de Arquivos (Volume Landing)
/Volumes/landing/pedidos/

ops.bronze.pedidos (Delta Table)

- Auto Loader
- Arquivos brutos
- Histórico completo

ops.silver.pedidos_clean (Delta Table)

- Limpeza e validação
- Padronização
- Deduplicação

ops.gold.receita_diaria (Delta Table)

- Agregação diária
- KPIs de negócio
- Pronto p/ BI & ML

Checkpoint
Unity Catalog Volume

Schema Evolution
Unity Catalog Volume

Governança & Linhagem
Unity Catalog + Delta

BOAS PRÁTICAS

Sempre use volumes do Unity Catalog para landing, checkpoints e schemas.

Separe ambientes: dev / stg / prd.

Mantenha Bronze imutável.

Documente contratos e regras de negócio.

Monitore latência, volume e falhas.

ASSETS DO LAB

Volume Landing:
/Volumes/landing/pedidos/

Volume Checkpoints:
/Volumes/checkpoints/pedidos_bronze

Volume Schemas:
/Volumes/schemas/pedidos_bronze

Tabelas: bronze, silver, gold

PRÓXIMO PASSO

Dia 5

ETL vs ELT e transformação

Bronze / Silver / Gold

Objetivo: Transformar a Bronze populada no Dia 4 em Silver limpa, tipada e deduplicada; aplicar MERGE/CDC quando necessário; e publicar Gold com métricas analíticas para consumo de negócio.

Narrativa: no Dia 4 os dados chegaram à Bronze. No Dia 5 eles passam por regras, validações e agregações até virarem ativos prontos para análise.

Jornada prática do laboratório

1 Recap Bronze

Validar bronze.pedidos_autoloader criada no Dia 4 e entender granularidade.

2 ETL vs ELT

Conectar teoria com padrão Lakehouse: carrega primeiro, transforma depois.

3 Bronze → Silver

Tipar, limpar, padronizar e deduplicar pedidos.

4 Validação Silver

Checar duplicidade, nulos críticos, quantidades inválidas e tipos.

5 CDC e MERGE

Aplicar upsert quando houver inserts, updates e deletes em Bronze CDC.

6 Silver → Gold

Criar receita_diaria e top_produtos como tabelas de consumo.

7 Reprocessamento

Recriar Silver/Gold a partir da Bronze quando regra mudar.

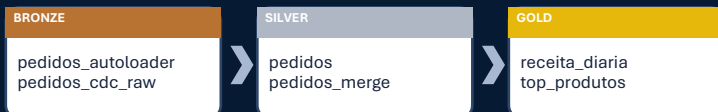
8 Ponte Dia 6

Preparar fatos, dimensões e mini data mart para analytics.

Entregável final

capgemini_academy

Bronze → Gold



Checklist de validação

- ✓ Bronze correta: pedidos_autoloader
- ✓ Silver sem duplicidade por pedido_id
- ✓ Nulos e quantidades inválidas tratados
- ✓ MERGE/CDC discutido ou executado
- ✓ Gold receita_diaria criada
- ✓ Gold top_produtos criada

Ajustes importantes nos slides

```

FROM bronze.pedidos_autoloader
CREATE OR REPLACE TABLE silver.pedidos_autoloader AS ...
MERGE INTO silver.pedidos USING bronze.pedidos_cdc_raw
CREATE OR REPLACE TABLE gold.receita_diaria AS ...
CREATE OR REPLACE TABLE gold.top_produtos AS ...
  
```

Critério: cada transformação precisa de regra explícita e validação objetiva.

A bronze preserva o passado. A silver organiza o presente. A gold entrega valor ao negócio.

Setup do laboratório

Consultar a Bronze do Dia 4.

```
USE CATALOG capgemini_academy;  
  
SELECT COUNT(*) AS total_bronze  
FROM capgemini_academy.bronze.pedidos_autoloader;  
  
SELECT *  
FROM capgemini_academy.bronze.pedidos_autoloader  
LIMIT 10;
```

Lab Parte 1: Bronze → Silver

Criar tabela limpa, deduplicada e tipada.

```
CREATE OR REPLACE TABLE capgemini_academy.silver.pedidos_autoloader AS
WITH ranked AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY pedido_id ORDER BY _ingested_at DESC) AS rn
    FROM capgemini_academy.bronze.pedidos_autoloader
)
SELECT
    CAST(pedido_id AS BIGINT) AS pedido_id,
    TRIM(cliente_id) AS cliente_id,
    TRIM(produto_id) AS produto_id,
    CAST(quantidade AS INT) AS quantidade,
    CAST(valor_total AS DECIMAL(10,2)) AS valor_total,
    TO_TIMESTAMP(data_pedido) AS data_pedido,
    CURRENT_TIMESTAMP() AS silver_ingest_ts
FROM ranked
WHERE rn = 1
    AND pedido_id IS NOT NULL
    AND quantidade > 0;
```

Lab Parte 2: validações da Silver

Não aceite a transformação sem evidência.

```
SELECT COUNT(*) AS total_silver  
FROM capgemini_academy.silver.pedidos_autoloader;
```

```
SELECT pedido_id, COUNT(*) AS qtd  
FROM capgemini_academy.silver.pedidos_autoloader  
GROUP BY pedido_id  
HAVING COUNT(*) > 1;
```

```
SELECT *  
FROM capgemini_academy.silver.pedidos_autoloader  
WHERE pedido_id IS NULL OR quantidade <= 0;
```

Validação 1

Sem duplicidade por pedido_id

Validação 2

Sem nulos críticos

Validação 3

Tipos e regras coerentes

Lab Parte 3: Silver → Gold

Publicar tabela analítica de receita diária.

```
CREATE OR REPLACE TABLE capgemini_academy.gold.receita_diaria AS
SELECT
  DATE(data_pedido) AS data_pedido,
  COUNT(DISTINCT pedido_id) AS pedidos,
  SUM(quantidade) AS itens_vendidos,
  SUM(valor_total) AS receita_total,
  ROUND(SUM(valor_total) / NULLIF(COUNT(DISTINCT pedido_id),0), 2) AS ticket_medio
FROM capgemini_academy.silver.pedidos_autoloader
GROUP BY DATE(data_pedido);

SELECT *
FROM capgemini_academy.gold.receita_diaria
ORDER BY data_pedido;
```

Lab Parte 3: Silver → Gold

Publicar tabela analítica de TOP produtos.

```
CREATE OR REPLACE TABLE capgemini_academy.gold.top_produtos AS
SELECT
    produto_id,
    SUM(quantidade) AS qtde_vendida,
    SUM(valor_total) AS receita
FROM capgemini_academy.silver.pedidos_autoloader
GROUP BY produto_id;

SELECT *
FROM capgemini_academy.gold.top_produtos
ORDER BY receita DESC;
```

Quiz rápido

Verifique se o aluno entendeu decisões, não só comandos.

- 1) Qual a diferença prática entre ETL e ELT?
- 2) Por que Bronze deve preservar dados brutos?
- 3) Quais transformações pertencem à Silver?
- 4) O que caracteriza uma tabela Gold?
- 5) Quando usar MERGE em vez de CREATE OR REPLACE?
- 6) Como você reproprocessaria Gold se uma regra de negócio mudasse?

Erros comuns a evitar

Esses erros viram dívida técnica rapidamente.

Erro 1

Aplicar regra de negócio definitiva na Bronze.

Erro 2

Criar Gold sem validação da Silver.

Erro 3

Não documentar regras de transformação.

Erro 4

Não controlar deduplicação por chave.

Erro 5

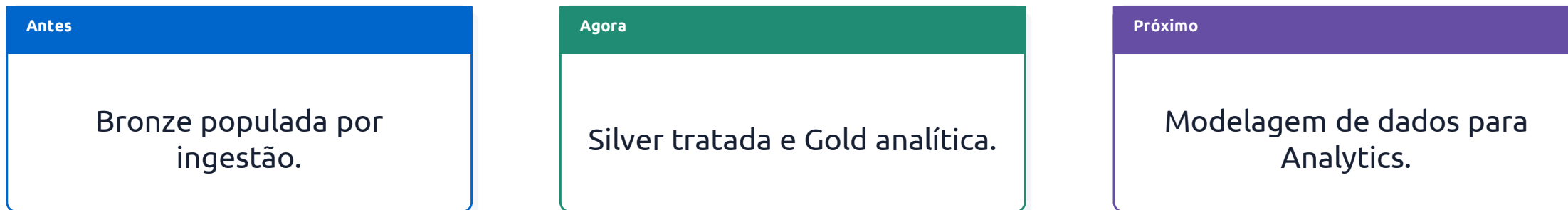
Usar append quando deveria ser upsert.

Erro 6

Não planejar reproprocessamento.

Ponte para o Dia 6

Agora os dados deixaram de ser apenas registros operacionais. Eles foram transformados em ativos analíticos.



Entrar no Kaggle e buscar os datasets:

- Retail Sales Dataset
- **Indian E-commerce Sales Analytics (obrigatório)**
- Banking Transactions
- Retail Data
- FIFA World Cup 2026: Players + Matches

No Dia 6 aprenderemos a estruturar esses ativos em fatos dimensões e modelos de consume.



Desafio do Dia 5 – Assistir e resumir

- Data Warehouse vs Data Lake vs Lakehouse
<https://www.youtube.com/watch?v=Grx95nfsz08>

Thank you.

Make it real.

Capgemini 